



Universidad Tecnológica Nacional
Facultad Regional Córdoba

Algoritmos y Estructuras de Datos

Ficha Nro. 18

Registros

Prof. Corso Cynthia

Temario

- Introducción
- Registros en Python
 - Concepto. Estructura
 - Creación e Inicialización de registros mediante constructores
- Arreglo de Registros

Introducción

- **Estructura de datos:** se define como una variable capaz de contener varios valores al mismo tiempo. Algunos ejemplos ya estudiados: tuplas, cadenas de caracteres, rangos y listas.
- Es una situación común que el programador necesite almacenar valores de diferentes tipos en una misma estructura de datos.
- Una alternativa es incorporar el uso de **registros**.

Registros en Python

- Un **registro** es un conjunto mutable de valores que pueden ser de distintos tipos.
- Cada componente de un registro se denomina **campo** (o también atributo).
- **Ventaja de uso:** son útiles para representar en forma clara a cualquier entidad del mundo real que el programador necesite manejar.



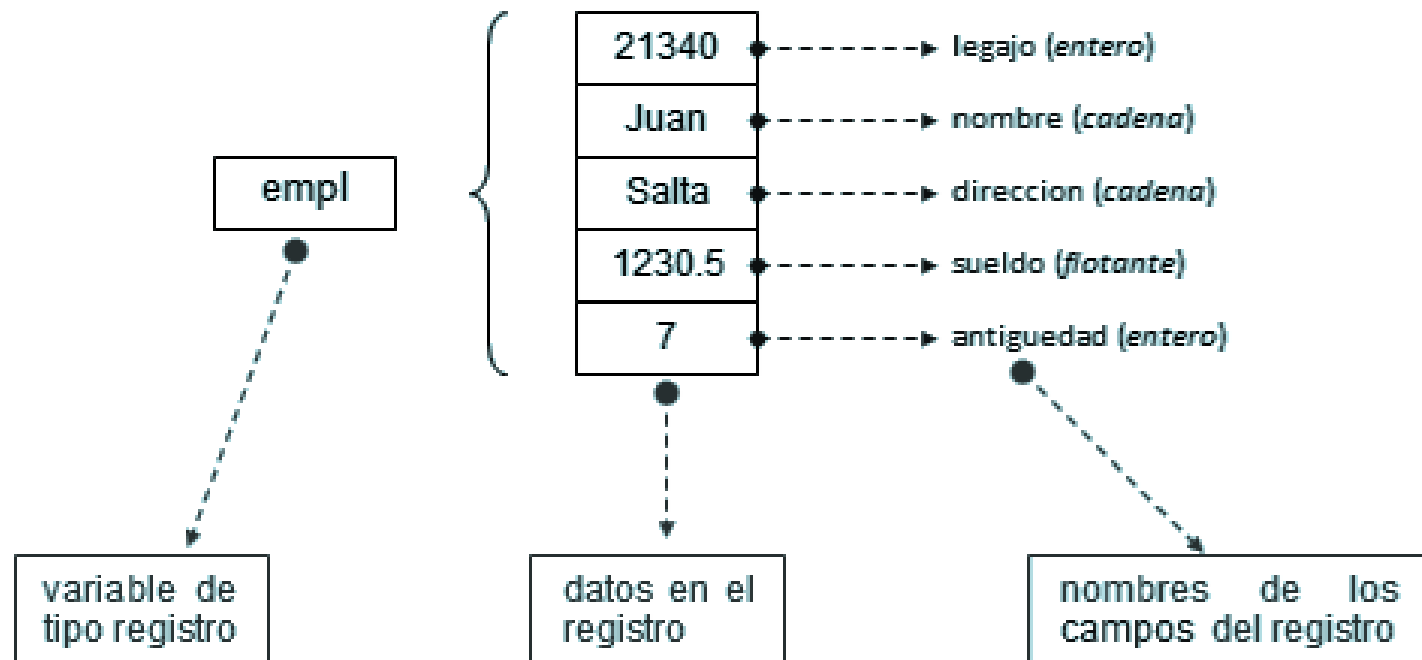
Ejemplo de uso de “registro” en Python

- Caso de estudio: se solicita desarrollar un programa que almacene la información de un empleado de una empresa comercial, y se supone que se necesita del “empleado” almacenar los siguientes datos:
 - Legajo
 - Nombre
 - Dirección
 - Sueldo
 - Antigüedad
- En este contexto es posible utilizar una **variable** de tipo **registro**.

Variables de tipo registro: Estructura

- Se puede almacenar en una misma variable **todos los datos del empleado** (o elementos del dominio del problema), como se muestra en la Figura 1.

Figura 1: Esquema de una variable de tipo registro.



Implementación en Python de un registro

- Los pasos a seguir para implementar un registro en Python son:
 - Declarar un nombre o identificador de tipo de dato para **registro**. La sintaxis en Python es la siguiente:

```
class Empleado:  
    pass
```

- Crear una variable del tipo registro (en este caso del tipo Empleado).
- Finalmente se definen las variables (campos) para ese nuevo tipo de dato.

Representación gráfica de una variable tipo registro “Empleado”

```
class Empleado:
    pass
```

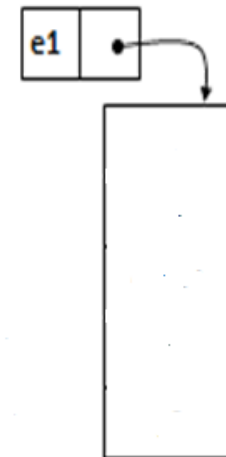
```
e1 = Empleado() • →
```

↑

1) Se crea una variable registro tipo **Empleado** (vacía) mediante la función constructora **Empleado()** disponible para los tipos creados por la palabra class.

2) Lo que se almacena en **e1** es una **referencia o dirección de memoria** (hexadecimal).
Ejemplo: e1 podría almacenar B7B (hexadecimal)

En este caso la variable **e1** se crea una variable registro tipo **Empleado**, por ahora vacía (sin ningún campo).



B7B en hexadecimal

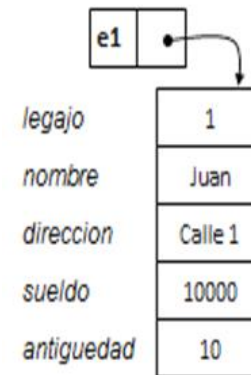
Incorporación de campos a la variable registro tipo “Empleado”

```
class Empleado:  
    pass
```

```
e1 = Empleado()  
e1.legajo = 1  
e1.nombre = 'Juan'  
e1.direccion = 'Calle 1'  
e1.sueldo = 10000  
e1.antiguedad = 10
```

La variable **e1** se crea como un registro de tipo **Empleado** “vacío”.

Para agregar los “campos” debe usarse el **operador punto**.



B7B en hexadecimal

Acceso a los campos de una variable de tipo registro

- La forma de acceso a los campos también se realiza mediante el operador punto (.).
 - Se escribe el nombre de la variable tipo registro.
 - Seguido de punto, y.
 - El nombre de campo al cual se necesita incorporar o acceder.

Ejemplo:

```
print('Dirección', e1.direccion)
```

Ejemplo de creación de otras variables registro tipo Empleado

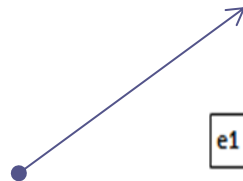
```
class Empleado:
    pass
```

```
e2 = Empleado()
e2.legajo = 2
e2.nombre = 'Juan'
e2.direccion = 'Calle 2'
e2.sueldo = 20000
e2.antiguedad = 15
```

```
e3 = Empleado()
e3.legajo = 3
e3.nombre = 'Pedro'
e3.direccion = 'Calle 3'
e3.sueldo = 25000
e3.antiguedad = 20
```



Las variables **e1**, **e2** y **e3** se crean como un registro de tipo **Empleado**.
Las variables **e1**, **e2**, **e3** representan una referencia.



e1	•
legajo	1
nombre	Juan
direccion	Calle 1
sueldo	10000
antiguedad	10

e2	•
legajo	2
nombre	Luis
direccion	Calle 2
sueldo	20000
antiguedad	15

e3	•
legajo	3
nombre	Pedro
direccion	Calle 3
sueldo	25000
antiguedad	20

Visualización de los campos de las variables registro tipo Empleado. Ejemplo..

```
#Visualización del campo legajo y sueldo..
```

```
print('Empleado 1 -Legajo: ', e1.legajo, ' Sueldo: ', e1.sueldo)
```

```
print('Empleado 2 -Legajo: ', e2.legajo, ' Sueldo: ', e2.sueldo)
```

```
print('Empleado 3 -Legajo: ', e3.legajo, ' Sueldo: ', e3.sueldo)
```

Otras operaciones válidas con variables de tipo registro

- Como un registro es una estructura **mutable** es posible realizar las siguientes operaciones:

```
# cambiar el legajo de e1...
e1.legajo = 4

# hacer que la antigüedad de e3 sea igual que la de e2...
e3.antigüedad = e2.antigüedad

# cargar por teclado el sueldo de e2...
e2.sueldo = float(input('Ingrese el nuevo sueldo del empleado 2: '))

# mostrar la dirección del empleado e1...
print('Dirección del empleado 1:', e1.dirección)

# sumar un año a la antigüedad de e3...
e3.antigüedad += 1
```

Cuestiones a tener en cuenta con el manejo de registros en Python

- En Python las variables no quedan atadas a un tipo fijo y estático, nada impediría que se cree una cuarta variable tipo **Empleado**, pero con una estructura de campos diferentes a las tres que ya existen.

```
# creación de una variable Empleado con un campo
# adicional...
e4 = Empleado()
e4.legajo = 10
e4.nombre = 'Luis'
e4.direccion = 'Calle 4'
e4.sueldo = 50000
e4.antiguedad = 30
e4.cargo = 'Gerente'
```

Cuestiones a tener en cuenta con el manejo de registros en Python

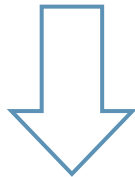
- La siguiente secuencia de instrucciones muestra los legajos y nombre de los cuatro empleados, mas el cargo del cuarto empleado.

```
# mostrar datos básicos...
print('E1 - Legajo:', e1.legajo, '- Nombre:', e1.nombre)
print('E2 - Legajo:', e2.legajo, '- Nombre:', e2.nombre)
print('E3 - Legajo:', e3.legajo, '- Nombre:', e3.nombre)
print('E4 - Legajo:', e4.legajo, '- Nombre:', e4.nombre, '-
Cargo:', e4.cargo)
```

Cuestiones a tener en cuenta con el manejo de registros en Python

- La siguiente línea de código provocaría un error:

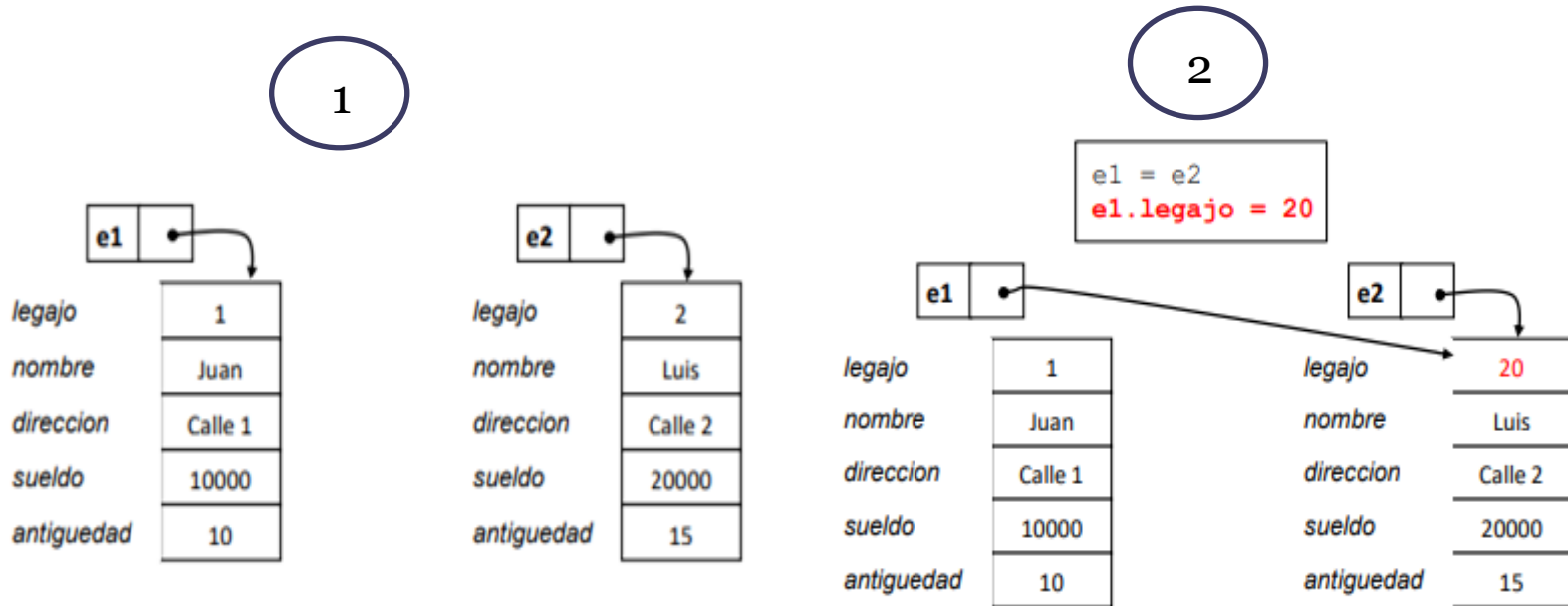
```
print('E3-Legajo:', e3.legajo, '-Nombre:', e3.nombre, '-  
Cargo:', e3.cargo)
```



```
File "C:/Ejemplo/test02.py", line 42, in <module>  
    print('E3 - Legajo:', e3.legajo, '- Nombre:', e3.nombre,  
    '- Cargo:', e3.cargo)  
AttributeError: 'Empleado' object has no attribute 'cargo'
```

Analicemos esta situación con las variables tipo registro

- **Recordar!!!** que las variables tipo registro **e1** y **e2** almacenan una referencia (dirección de memoria) que apunta a la estructura registro (Figura 1).
- Si se hace una operación de asignación entre dos variables tipo registro (como en la Figura 2), **no se hace una copia del registro original, sino una copia de las referencias..**



Inicialización de variables de tipo registro

```
class Empleado:  
    pass
```

```
e2 = Empleado()  
e2.legajo = 2  
e2.nombre = 'Juan'  
e2.direccion = 'Calle 2'  
e2.sueldo = 20000  
e2.antiguedad = 15
```

```
e3 = Empleado()  
e3.legajo = 3  
e3.nombre = 'Pedro'  
e3.direccion = 'Calle 3'  
e3.sueldo = 25000  
e3.antiguedad = 20
```

La tarea de la inicialización del registro la podemos hacer de manera mas compacta mediante la implementación de una función que se encargue de dicha tarea.



Creación e inicialización de registros mediante constructores

```
# declaración de una clase con un constructor...
class Empleado:
    # un constructor dentro de la clase Empleado...
    def __init__(self, leg, nom, direc, suel, ant):
        self.legajo = leg
        self.nombre = nom
        self.direccion = direc
        self.sueldo = suel
        self.antiguedad = ant
```



legajo	1
nombre	Juan
direccion	Calle 1
sueldo	10000
antigüedad	10

`self` queda asignado con la dirección del registro que se está creando.

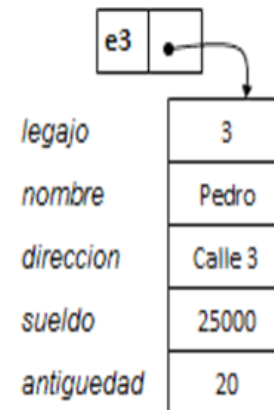
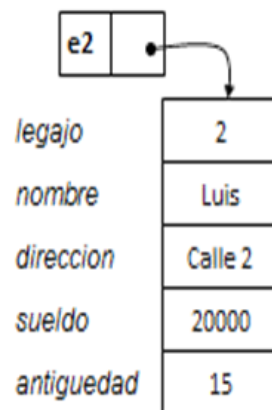
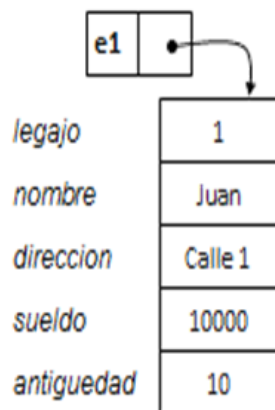
#Creación de registro e1 mediante constructor..

```
e1 = Empleado(1, 'Juan', 'Calle 1', 10000,10)
```

La parte derecha de esta asignación invoca al constructor `__init__()` de la clase `Empleado`.

Creación e inicialización de registros mediante constructores

```
# una funcion de prueba
def test():
    # creación e inicialización de objetos de la clase Empleado...
    e1 = Empleado(1, 'Juan', 'Calle 1', 10000, 10)
    e2 = Empleado(2, 'Luis', 'Calle 2', 20000, 15)
    e3 = Empleado(3, 'Pedro', 'Calle 3', 25000, 20)
```



Visualización de los datos de un registro

```
# una función para mostrar un objeto o instancia de la clase Empleado
def write(emplado):
    print("\nLegajo:", emplado.legajo, end=' ')
    print("- Nombre:", emplado.nombre, end=' ')
    print("- Direccion:", emplado.direccion, end=' ')
    print("- Sueldo:", emplado.sueldo, end=' ')
    print("- Antigüedad:", emplado.antigüedad, end=' ')
```

Esta función permite la visualización de todos los campos que tiene el registro Empleado.

```
# una funcion de prueba
def test():
    # creación e inicialización de objetos de la clase Empleado...
    e1 = Empleado(1, 'Juan', 'Calle 1', 10000, 10)
    e2 = Empleado(2, 'Luis', 'Calle 2', 20000, 15)
    e3 = Empleado(3, 'Pedro', 'Calle 3', 25000, 20)

    # visualizacion de los valores de los tres objetos...
    write(e1)
    write(e2)
    write(e3)
```

Arreglos de registros. Caso de estudio

Enunciado: Se desea almacenar en un **arreglo** la información de los “n” **estudiantes** que se registraron para participar de un curso de programación.

Por cada estudiante se tiene los siguientes datos:

- su número de legajo,
- su nombre y
- su promedio en la carrera que cursa.

Muestre los legajos de los estudiantes del curso, que cumplan esta condición: que su promedio sea mayor o igual a 8.

Pasos para resolver el caso de estudio

1) Definir el registro que pertenece a la entidad que se desea almacenar en el arreglo.

Por ejemplo: si se desea almacenar “estudiantes” en el arreglo se debe tener la definición del registro **Estudiante** por medio de la función constructora **__init__()**.

```
class Estudiante:  
    def __init__(self, leg, nom, pro):  
        self.legajo = leg  
        self.nombre = nom  
        self.promedio = pro
```

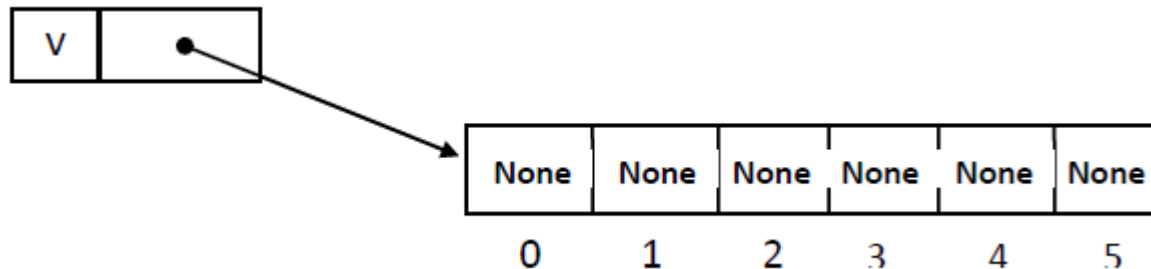
Pasos para resolver el caso de estudio

2) Si se conoce de antemano la cantidad de “n” de registros a almacenar, una forma de crear el arreglo es declarar la referencia de ese arreglo reservando “n” casilleros con cualquier valor, por ejemplo **None**.

```
n = 6
```

```
v = n * [None]
```

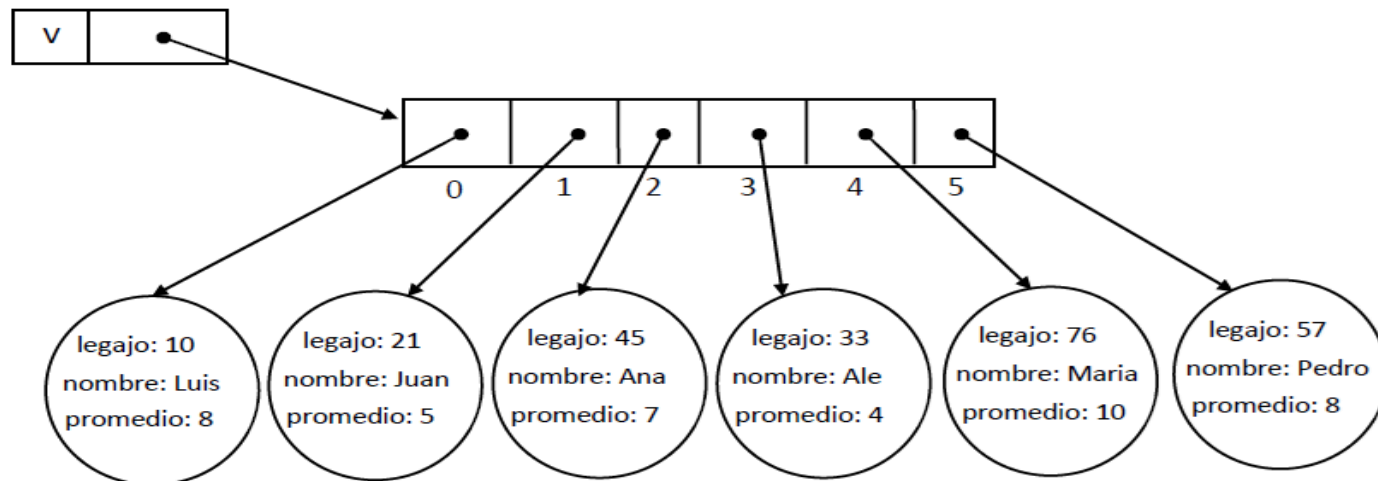
Esta última instrucción crea un arreglo de $n=6$ componentes con valor inicial **None**.



Pasos para resolver el caso de estudio

3) Creación de registros (Estudiantes): estos serán apuntados desde cada casilla del arreglo y se podrá comenzar a trabajar con esos objetos. Una alternativa de carga podría ser:

```
for i in range(n):
    leg = int(input('Legajo: '))
    nom = input('Nombre: ')
    pro = float(input('Promedio: '))
    v[i] = Estudiante(leg, nom, pro)
```



Pasos para resolver el caso de estudio

4) Procesamiento del arreglo: se usan las técnicas de recorridos ya estudiadas. Si se conoce que clase de registro se almacenó en el arreglo se podrá visualizarlo su contenido por teclado, ordenarlo, efectuar búsquedas etc.

```
# Se asume que el arreglo ya esta cargado.  
# Se efectúa un recorrido con filtro.  
for i in range(len(v)):  
    if v[i].promedio > 8:  
        print('Legajo alumno',v[i].legajo)
```



Este script visualizará los legajos de los estudiantes con promedio superior a 8.

Muchas Gracias por la atención!!

Cualquier duda pueden contactarme por mensajería interna del Aula Virtual.